

SHARIF
UNIVERSITY OF TECHNOLOGY



Computer Structure

Pre-Lab Report 4

Aria Fateh Kerdari
403110382

July 1, 2026

1 Matrix Keyboard Scanning

1.1 How it Works

At each intersection of a row and a column, there is a push-button switch. When open, the row and column are physically disconnected. When pressed, the switch bridges the specific row and column together.

To figure out which button is pressed, the microcontroller tests one row at a time by driving it to a specific logic level (e.g., LOW). It then reads the state of all the columns. If a column reads LOW, the microcontroller knows that the switch at the intersection of the currently active row and that specific column is closed.

1.2 Debounce

We use software debouncing. When the code detects a change in a pin's state, it registers a timestamp. We then mandate a short period (usually 20 to 50 milliseconds) where any further changes on that pin are ignored. Then we trust the new state.

1.3 Short-Circuit Hazard

Normally, matrix rows are driven using the GPIOs in the output direction. Imagine Row 1 is driven **HIGH** and Row 2 is driven **LOW**. If you simultaneously press a button on Row 1 and a button on Row 2 in the same column, you create a direct wire connection between a pin outputting 3.3V and a pin outputting 0V.

By completely forbidding the use of the **OUT** direction, you keep all pins in a "High-Impedance" state. High-impedance pins cannot source or sink dangerous amounts of current.

Columns are set to **IN** with a **PULL_UP** resistor. Inactive Rows are set to **IN** with **PULL_UP** resistor. The Active Scanning Row is set to **IN** with a **PULL_DOWN** resistor. When a button is pressed, it bridges the column (3.3 V) and the active row (0 V).

Because the RP2040's internal pull resistors are relatively big, connecting a pull-up to a pull-down only allows a microscopic current to flow. This amount of current doesn't burn a pin.

1.4 Code

Complete project is available in this link.

2 Alarm Functionality

2.1 Repeating Alarm and Simple Delay

The fundamental difference of repeating alarm (`machine.Timer` in micro-python) and simple delay (`time.sleep`) comes down to blocking vs. non-blocking behavior.

When processor gets to `time.sleep()` , it halts execution of your current thread. Your program is "blocked" from advancing.

A repeating alarm offloads the counting task to a dedicated hardware timer peripheral inside the processor. You tell the hardware timer, trigger this specific function every `t` seconds. The main CPU immediately moves on to execute the rest of your code. When the hardware timer finishes counting, tells CPU to execute the callback function, and then the CPU goes right back to whatever it was doing.

2.2 When to Prefer a Repeating Alarm Over Polling

A repeating alarm uses hardware to guarantee the callback fires at exact, precise intervals. Simple delay is highly inefficient because CPU is running at full capacity just to ask "is it time yet?" millions of times in a second. While a repeating alarm can put processor into low-power sleep and the low-power hardware timer counts seconds. The timer only wakes up the CPU when it is time to execute callback function.

3 PWM-LED

When you initialize a Raspberry Pi Pico using the official C/C++ SDK, the default system clock (`sys_clk`) frequency is 125 MHz. The project code is in this link.

4 PWM-Sine wave

4.1 Low-pass filter

A microcontroller's PWM pin does not output a true analog voltage; it only outputs a fast digital square wave altering between 0V and 3.3V. The low-pass filter, filters high frequencies from passing, so it gives a smother curve that is much more closer to a sine wave than what we had first.

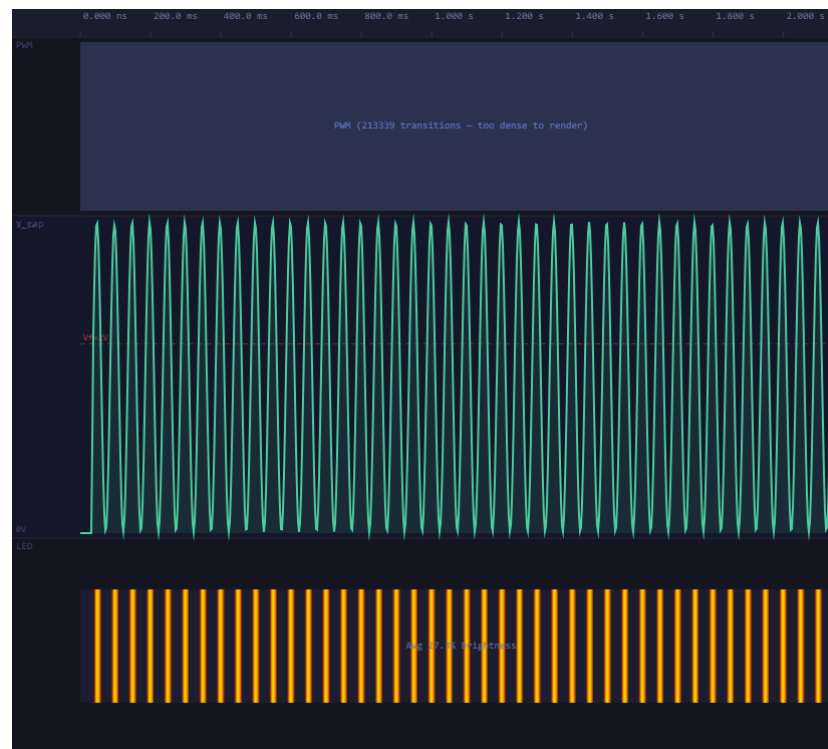
4.2 Role of Repeating Alarm

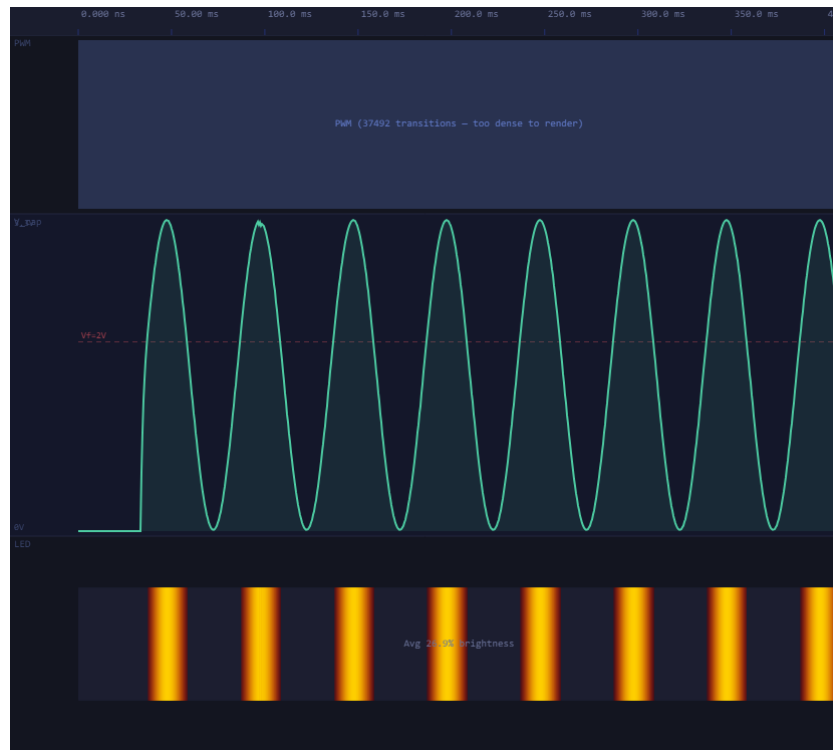
To generate a 20Hz sine wave, the duty cycle of your PWM pin must constantly and precisely change to trace the shape of the wave.

The repeating alarm (a hardware timer) acts as the sampling clock. It guarantees that the PWM duty cycle is updated at rigidly exact intervals (e.g., exactly every 1 millisecond).

4.3 Result

The result after applying a RLC filter is shown below. You can access the project using this link.





4.4 High PWM Frequency

$$\Delta V_{pp} \approx \frac{V_{cc}}{4f_{pwm}RC}$$

Look at the denominator in the ripple formula above: $4f_{pwm}RC$. As f_{pwm} gets larger, the ripple voltage gets small. The capacitor simply does not have enough time to charge or discharge between the fast pulses, resulting in a near-perfectly analog curve.